

TASK 14: ASSIGNMENT NO. 14

Name of Student : Anuja Vilas Wankhade

Domain: Artificial intelligence and machine Learning (AIML)

College Name: Manav School of Polytechnic AKOLA

Branch : Computer Engineering

GitHub Repositories Link : <https://github.com/anujaw193-star/python-project-collection>

Q1. (Array Properties)

PROGRAM CODE :

```
import numpy as np
# Create a 2D array of shape (5, 6) with random integers between 10 and 100
arr = np.random.randint(10, 101, size=(5, 6))
# Print the array
print("Array:")
print(arr)
# Print properties
print("\nShape of the array:", arr.shape)
print("Total number of elements (size):", arr.size)
print("Data type (dtype):", arr.dtype)
```

OUTPUT :

```
PS C:\Users\anuja\OneDrive\Desktop\ITR AIML> Python Task14.py
Array:
[[ 45  88  71  65  59 100]
 [ 61  47  91  37  35  56]
 [ 29  27  55  30  55  84]
 [ 84  75  37  39  56  29]
 [ 17  21  31  13  82  90]]

Shape of the array: (5, 6)
Total number of elements (size): 30
Data type (dtype): int32
PS C:\Users\anuja\OneDrive\Desktop\ITR AIML> █
```

Q2. (Statistical Methods – Basic)

PROGRAM CODE :

```
import numpy as np
# Generate a 1D array of 20 random numbers between 1 and 50
arr = np.random.randint(1, 51, size=20)
```

```

print("Array:")
print(arr)
# Statistical methods
print("\nMinimum value:", arr.min())
print("Index of minimum value:", arr.argmin())
print("\nMaximum value:", arr.max())
print("Index of maximum value:", arr.argmax())
print("\nSum of all elements:", arr.sum())
print("Mean:", arr.mean())
print("Standard Deviation:", arr.std())

```

OUTPUT :

```

PS C:\Users\anuja\OneDrive\Desktop\ITR AIML> Python Task14.py
Array:
[38 12 34 39 19 18  9 47 49 34 35 31 46 23 25 42 30  6 25  2]

Minimum value: 2
Index of minimum value: 19

Maximum value: 49
Index of maximum value: 8

Sum of all elements: 564
Mean: 28.2
Standard Deviation: 13.55949851580065
PS C:\Users\anuja\OneDrive\Desktop\ITR AIML>

```

Q3. (Statistical Methods on 2D Array)

PROGRAM CODE :

```

import numpy as np

# Create a 4x5 matrix of random integers between 20 and 80
matrix = np.random.randint(20, 81, size=(4, 5))
print("Matrix:")
print(matrix)
# Statistics for the entire matrix
print("\nMinimum value:", matrix.min())
print("Maximum value:", matrix.max())
print("Sum of all elements:", matrix.sum())
print("Mean:", matrix.mean())
print("Standard Deviation:", matrix.std())
# Sum of each row
print("\nSum of each row:")
print(matrix.sum(axis=1))
# Sum of each column
print("\nSum of each column:")

```

```
print(matrix.sum(axis=0))
```

OUTPUT :

```
PS C:\Users\anuja\OneDrive\Desktop\ITR AIML> Python Task14.py
Matrix:
[[58 36 68 43 54]
 [54 42 80 65 60]
 [30 35 43 27 74]
 [23 55 65 45 80]]

Minimum value: 23
Maximum value: 80
Sum of all elements: 1037
Mean: 51.85
Standard Deviation: 16.686146948891466

Sum of each row:
[259 301 209 268]

Sum of each column:
[165 168 256 180 268]
PS C:\Users\anuja\OneDrive\Desktop\ITR AIML> |
```

Q4. (Reshape)

PROGRAM CODE :

```
import numpy as np
# Create a 1D array containing numbers from 1 to 24
arr = np.arange(1, 25)
print("Original 1D Array:")
print(arr)
print("Shape:", arr.shape)
# Reshape into (4, 6)
arr_4x6 = arr.reshape(4, 6)
print("\nArray reshaped to (4, 6):")
print(arr_4x6)
print("Shape:", arr_4x6.shape)
# Reshape into (3, 8)
arr_3x8 = arr.reshape(3, 8)
print("\nArray reshaped to (3, 8):")
print(arr_3x8)
print("Shape:", arr_3x8.shape)
# Reshape into (2, 3, 4)
arr_3d = arr.reshape(2, 3, 4)
print("\nArray reshaped to (2, 3, 4):")
print(arr_3d)
print("Shape:", arr_3d.shape)
```

OUTPUT :

```
PS C:\Users\anuja\OneDrive\Desktop\ITR AIML> Python Task14.py
Original 1D Array:
[ 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24]
Shape: (24,)

Array reshaped to (4, 6):
[[ 1  2  3  4  5  6]
 [ 7  8  9 10 11 12]
 [13 14 15 16 17 18]
 [19 20 21 22 23 24]]
Shape: (4, 6)

Array reshaped to (3, 8):
[[ 1  2  3  4  5  6  7  8]
 [ 9 10 11 12 13 14 15 16]
 [17 18 19 20 21 22 23 24]]
Shape: (3, 8)

Array reshaped to (2, 3, 4):
[[[ 1  2  3  4]
   [ 5  6  7  8]
   [ 9 10 11 12]]
 [[13 14 15 16]
   [17 18 19 20]
   [21 22 23 24]]]
Shape: (2, 3, 4)
PS C:\Users\anuja\OneDrive\Desktop\ITR AIML>
```

Q5.(NumPy Indexing – 1D & 2D)

PROGRAM CODE :

```
import numpy as np
# Create the array
arr = np.array([
    [10, 20, 30, 40],
    [50, 60, 70, 80],
    [90, 100, 110, 120]
])
print("Original Array:")
print(arr)
# Extract first row
print("\nFirst Row:")
print(arr[0])
# Extract last column
print("\nLast Column:")
print(arr[:, -1])
# Extract center 2x2 submatrix
print("\nCenter 2x2 Submatrix:")
print(arr[1:3, 1:3])
# Extract all even numbers using boolean indexing
print("\nAll Even Numbers:")
print(arr[arr % 2 == 0])
```

OUTPUT :

```
PS C:\Users\anuja\OneDrive\Desktop\ITR AIML> Python Task14.py
Original Array:
[[ 10  20  30  40]
 [ 50  60  70  80]
 [ 90 100 110 120]]

First Row:
[10 20 30 40]

Last Column:
[ 40  80 120]

Center 2x2 Submatrix:
[[ 60  70]
 [100 110]]

All Even Numbers:
[ 10  20  30  40  50  60  70  80  90 100 110 120]
PS C:\Users\anuja\OneDrive\Desktop\ITR AIML>
```

Q6.(Advanced Indexing)

PROGRAM CODE :

```
import numpy as np
# Create a 5x5 matrix of random integers between 1 and 100
arr = np.random.randint(1, 101, size=(5, 5))
print("Original Matrix:")
print(arr)
# Print diagonal elements
print("\nDiagonal Elements:")
print(np.diag(arr))
# Print elements greater than 50
print("\nElements Greater Than 50:")
print(arr[arr > 50])
# Replace all elements less than 30 with 0
arr[arr < 30] = 0
# Print modified array
print("\nModified Matrix (elements < 30 replaced with 0):")
print(arr)
```

OUTPUT :

```

PS C:\Users\anuja\OneDrive\Desktop\ITR AIML> Python Task14.py
Original Matrix:
[[39 18 76 17 92]
 [17 88 89 32  5]
 [81 81 65 63 27]
 [84 19 62 40 99]
 [83 62 17 18 73]]

Diagonal Elements:
[39 88 65 40 73]

Elements Greater Than 50:
[76 92 88 89 81 81 65 63 84 62 99 83 62 73]

Modified Matrix (elements < 30 replaced with 0):
[[39  0 76  0 92]
 [ 0 88 89 32  0]
 [81 81 65 63  0]
 [84  0 62 40 99]
 [83 62  0  0 73]]
PS C:\Users\anuja\OneDrive\Desktop\ITR AIML>

```

Q7. (Arithmetic Operations)

PROGRAM CODE :

```

import numpy as np
# Create arrays
a = np.array([10, 20, 30, 40])
b = np.array([1, 2, 3, 4])
print("Array a:", a)
print("Array b:", b)
# Arithmetic Operations
print("\nAddition (a + b):")
print(a + b)
print("\nSubtraction (a - b):")
print(a - b)
print("\nMultiplication (a * b):")
print(a * b)
print("\nDivision (a / b):")
print(a / b)
print("\nElement-wise Power (a ** b):")
print(a ** b)
print("\nModulo Operation (a % b):")
print(a % b)

```

OUTPUT :

```

PS C:\Users\anuja\OneDrive\Desktop\ITR AIML> Python Task14.py
Array a: [10 20 30 40]
Array b: [1 2 3 4]

Addition (a + b):
[11 22 33 44]

Subtraction (a - b):
[ 9 18 27 36]

Multiplication (a * b):
[ 10 40 90 160]

Division (a / b):
[10. 10. 10. 10.]

Element-wise Power (a ** b):
[    10    400   27000 2560000]

Modulo Operation (a % b):
[0 0 0 0]
PS C:\Users\anuja\OneDrive\Desktop\ITR AIML>

```

Q8. (Matrix Multiplication)

PROGRAM CODE :

```

import numpy as np
# Create matrices
A = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])
B = np.array([[9, 8, 7],
              [6, 5, 4],
              [3, 2, 1]])
print("Matrix A:")
print(A)
print("\nMatrix B:")
print(B)
# Element-wise multiplication
element_wise = A * B
print("\nElement-wise Multiplication (A * B):")
print(element_wise)
# Matrix multiplication
matrix_mult = A @ B # or np.dot(A, B)
print("\nMatrix Multiplication (A @ B):")
print(matrix_mult)
# Difference:
# A * B multiplies corresponding elements at the same position.
# A @ B performs row-by-column matrix multiplication.

```

OUTPUT :

```
PS C:\Users\anuja\OneDrive\Desktop\ITR AIML> Python Task14.py
Matrix A:
[[1 2 3]
 [4 5 6]
 [7 8 9]]

Matrix B:
[[9 8 7]
 [6 5 4]
 [3 2 1]]

Element-wise Multiplication (A * B):
[[ 9 16 21]
 [24 25 24]
 [21 16  9]]

Matrix Multiplication (A @ B):
[[ 30  24  18]
 [ 84  69  54]
 [138 114  90]]
PS C:\Users\anuja\OneDrive\Desktop\ITR AIML>
```

Q9. (Combined – Properties + Operations + Indexing)

PROGRAM CODE :

```
import numpy as np
# Generate a 6x6 matrix of random numbers
arr = np.random.randn(6, 6)
print("Original Matrix:")
print(arr)
# Properties
print("\nShape:", arr.shape)
print("Size:", arr.size)
print("Data Type:", arr.dtype)
# Index of maximum and minimum value
print("\nIndex of Maximum Value:", np.unravel_index(arr.argmax(), arr.shape))
print("Index of Minimum Value:", np.unravel_index(arr.argmin(), arr.shape))
# Extract top-left 3x3 submatrix
print("\nTop-Left 3x3 Submatrix:")
print(arr[:3, :3])
# Replace negative numbers with their absolute value
arr[arr < 0] = np.abs(arr[arr < 0])
print("\nModified Matrix (Negative Values Replaced):")
print(arr)
# Mean of modified matrix
print("\nMean of Modified Matrix:", arr.mean())
```

OUTPUT :


```

PS C:\Users\anuja\OneDrive\Desktop\ITR AIMS> Python Task14.py
Original Matrix:
[[ 0.26592345  0.93904561 -2.00855563  0.92753223  0.81006672  0.73185886]
 [-0.44217953  0.15286554 -1.0356851  -0.62465291  1.71348322  0.02640949]
 [-0.22338187  0.08600612 -1.68149886  0.97679697  0.82831151 -1.15354403]
 [-0.86985254  0.21107515  0.03259957  0.1365583  -1.90710835 -1.30672117]
 [-0.81381708  2.34045023 -1.39389993  0.67710945  0.44807172  2.00013898]
 [ 1.05437757 -0.51553408  0.99508902  0.55541037  1.08513235  2.380854  ]]

Shape: (6, 6)
Size: 36
Data Type: float64

Index of Maximum Value: (np.int64(5), np.int64(5))
Index of Minimum Value: (np.int64(0), np.int64(2))

Top-Left 3x3 Submatrix:
[[ 0.26592345  0.93904561 -2.00855563]
 [-0.44217953  0.15286554 -1.0356851 ]
 [-0.22338187  0.08600612 -1.68149886]]

Modified Matrix (Negative Values Replaced):
[[0.26592345  0.93904561  2.00855563  0.92753223  0.81006672  0.73185886]
 [0.44217953  0.15286554  1.0356851  0.62465291  1.71348322  0.02640949]
 [0.22338187  0.08600612  1.68149886  0.97679697  0.82831151  1.15354403]
 [0.86985254  0.21107515  0.03259957  0.1365583  1.90710835  1.30672117]
 [0.81381708  2.34045023  1.39389993  0.67710945  0.44807172  2.00013898]
 [1.05437757  0.51553408  0.99508902  0.55541037  1.08513235  2.380854  ]]

Mean of Modified Matrix: 0.9264332648636174
PS C:\Users\anuja\OneDrive\Desktop\ITR AIMS>

```

Q10. (Mini Project – Student Performance Analysis)

PROGRAM CODE :

```

import numpy as np

# =====
# Student Performance Analysis using NumPy
# =====
# Generate marks for 10 students and 5 subjects
# Random marks between 30 and 100
marks = np.random.randint(30, 101, size=(10, 5))
print("Student Marks (10 Students x 5 Subjects):")
print(marks)
# -----
# Calculate total marks and average for each student
# -----
total_marks = marks.sum(axis=1)      # Sum of each row
average_marks = marks.mean(axis=1)   # Average of each row
print("\nTotal Marks of Each Student:")
print(total_marks)
print("\nAverage Marks of Each Student:")
print(average_marks)
# -----
# Find student with highest and lowest total marks

```

```

# -----
highest_student = total_marks.argmax()
lowest_student = total_marks.argmin()
print("\nStudent with Highest Total Marks:")
print("Student Index:", highest_student)
print("Total Marks:", total_marks[highest_student])
print("\nStudent with Lowest Total Marks:")
print("Student Index:", lowest_student)
print("Total Marks:", total_marks[lowest_student])
# -----
# Calculate overall class statistics
# -----
class_mean = marks.mean()
class_std = marks.std()

print("\nOverall Class Mean:", class_mean)
print("Overall Class Standard Deviation:", class_std)
# -----
# Find Top 3 Students based on total marks
# -----
top3_indices = np.argsort(total_marks)[-3:][::-1]
print("\nTop 3 Students (Indices):")
print(top3_indices)
# Extract marks of top 3 students using indexing
top3_marks = marks[top3_indices]
print("\nMarks of Top 3 Students:")
print(top3_marks)
# -----
# Reshape Example
# Convert 10x5 array into 5x10 array
# -----
reshaped_marks = marks.reshape(5, 10)
print("\nReshaped Array (5 x 10):")
print(reshaped_marks)
# -----
# Example of Indexing
# First student's marks
# -----
print("\nMarks of First Student:")
print(marks[0])
# Marks in Subject 1 for all students
print("\nMarks of All Students in Subject 1:")
print(marks[:, 0])
# Top 3 students' marks
print("\nMarks of Top 3 Students (Using Indexing):")
print(marks[top3_indices])

```

OUTPUT :

```

PS C:\Users\anuja\OneDrive\Desktop\ITR AITML> Python Task14
Student Marks (10 Students x 5 Subjects):
[[91 34 97 92 39]
 [39 59 93 78 40]
 [38 84 30 64 63]
 [50 50 70 99 65]
 [88 98 54 86 59]
 [34 82 39 95 75]
 [83 49 66 42 49]
 [49 77 30 65 88]
 [56 85 54 71 92]
 [99 51 35 79 64]]

Total Marks of Each Student:
[353 309 279 334 385 325 289 309 358 328]

Average Marks of Each Student:
[70.6 61.8 55.8 66.8 77. 65. 57.8 61.8 71.6 65.6]

Student with Highest Total Marks:
Student Index: 4
Total Marks: 385

Student with Lowest Total Marks:
Student Index: 2
Total Marks: 279

```

```

Student with Lowest Total Marks:
Student Index: 2
Total Marks: 279

Overall Class Mean: 65.38
Overall Class Standard Deviation: 21.343748499267882

Top 3 Students (Indices):
[4 8 0]

Marks of Top 3 Students:
[[88 98 54 86 59]
 [56 85 54 71 92]
 [91 34 97 92 39]]

Reshaped Array (5 x 10):
[[91 34 97 92 39 39 59 93 78 40]
 [38 84 30 64 63 50 50 70 99 65]
 [88 98 54 86 59 34 82 39 95 75]
 [83 49 66 42 49 49 77 30 65 88]
 [56 85 54 71 92 99 51 35 79 64]]

Marks of First Student:
[91 34 97 92 39]

Marks of All Students in Subject 1:
[91 39 38 50 88 34 83 49 56 99]

Marks of Top 3 Students (Using Indexing):
[[88 98 54 86 59]
 [56 85 54 71 92]
 [91 34 97 92 39]]
PS C:\Users\anuja\OneDrive\Desktop\ITR AITML>

```

